Conjunctive Normal Form (CNF) Converter for Propositional Logic Formulas

Fatemeh Abshang

Spring 2024–2025

Abstract

This document presents a Python implementation that converts propositional logic formulas into Conjunctive Normal Form (CNF). The system builds upon a binary tree parser to apply systematic transformations including implication elimination, double negation removal, De Morgan's laws, and distribution of disjunction over conjunction. The converter handles standard logical operators: negation ($\neg$), conjunction ($\wedge$), disjunction ($\vee$), and implication ($\rightarrow$), producing equivalent formulas in CNF.

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 Conjunctive Normal Form (CNF) . . . 3
1.2.1 Definition . . . 3
1.2.2 Examples . . . 3
1.3 Transformation Steps . . . 3

2 System Architecture **4**
2.1 Data Structures . . . 4
2.2 Operator Precedence . . . 4

3 Core Algorithms **4**
3.1 Step 1: Implication Elimination . . . 4
3.1.1 Transformation Rule . . . 4
3.1.2 Algorithm . . . 4
3.1.3 Example . . . 4
3.2 Step 2: Double Negation Removal . . . 5
3.2.1 Transformation Rule . . . 5
3.2.2 Algorithm . . . 5
3.2.3 Example . . . 5
3.3 Step 3: Applying De Morgan's Laws . . . 6
3.3.1 Transformation Rules . . . 6
3.3.2 Algorithm . . . 6
3.3.3 Examples . . . 6
3.4 Step 4: Distribution of OR over AND . . . 6

3.4.1 Transformation Rules . . . 6
3.4.2 Algorithm . . . 9
3.4.3 Examples . . . 9

4 Complete Conversion Process **9**
4.1 Main Algorithm . . . 9
4.2 Iteration Strategy . . . 9

5 Formula Output **9**
5.1 String Representation . . . 9
5.2 Parenthesization Rules . . . 11

6 Complete Examples **11**
6.1 Example 1: Simple Implication . . . 11
6.2 Example 2: Complex Formula . . . 12
6.3 Example 3: De Morgan's Application . . . 12
6.4 Example 4: Nested Formula . . . 12

7 Complexity Analysis **12**
7.1 Time Complexity . . . 12
7.2 Space Complexity . . . 13
7.3 Optimization Notes . . . 13

8 Usage Instructions **13**
8.1 Input Format . . . 13
8.2 Running the Program . . . 13
8.3 Example Input Files . . . 14

9 Correctness and Soundness **14**
9.1 Logical Equivalence . . . 14
9.2 Termination . . . 14

10 Limitations and Extensions **14**
10.1 Current Limitations . . . 14
10.2 Potential Enhancements . . . 15

11 Applications **15**

12 Conclusion **15**

A Complete Source Code **15**

1 Introduction

1.1 Project Overview

Conjunctive Normal Form (CNF) is a standardized representation of logical formulas consisting of a conjunction of clauses, where each clause is a disjunction of literals. This project implements a CNF converter that:

- Parses propositional logic formulas into binary trees
- Eliminates implication operators
- Removes double negations
- Applies De Morgan's laws to push negations inward
- Distributes disjunction over conjunction
- Outputs the equivalent formula in CNF

1.2 Conjunctive Normal Form (CNF)

1.2.1 Definition

A formula is in Conjunctive Normal Form if it is a conjunction of one or more clauses, where each clause is a disjunction of literals.

Structure:

$$(l_{11} \vee l_{12} \vee \ldots \vee l_{1n_1}) \wedge (l_{21} \vee l_{22} \vee \ldots \vee l_{2n_2}) \wedge \ldots \wedge (l_{m1} \vee l_{m2} \vee \ldots \vee l_{mn_m})$$

Where each l_{ij} is a literal (a propositional variable or its negation).

1.2.2 Examples

- $(p \vee q) \wedge (\neg p \vee r)$ is in CNF
- $(p \wedge q) \vee r$ is NOT in CNF (OR is outside AND)
- $\neg(p \vee q)$ is NOT in CNF (negation not pushed to literals)

1.3 Transformation Steps

Converting a formula to CNF requires four main steps:

1. **Eliminate Implications**: Replace $A \to B$ with $\neg A \vee B$
2. **Move Negations Inward**:
   - Remove double negations: $\neg\neg A \equiv A$
   - Apply De Morgan's laws:
     - $\neg(A \wedge B) \equiv \neg A \vee \neg B$
     - $\neg(A \vee B) \equiv \neg A \wedge \neg B$

3. **Distribute OR over AND**: Apply distributive law:
   - $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
   - $(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$
4. **Iterate**: Repeat steps 2-3 until no more changes occur

2 System Architecture

2.1 Data Structures

The system uses the same Node class from Phase 1:

```python
class Node:
    def __init__(self, key):
        self.data = key
        self.left: Node | None = None
        self.right: Node | None = None
```

Listing 1: Node class definition

2.2 Operator Precedence

Operator	Symbol	Precedence
Implication	$\rightarrow$	1 (lowest)
Disjunction	$\vee$	2
Conjunction	$\wedge$	3
Negation	$\neg$	4 (highest)

Table 1: Logical operators and their precedence levels

3 Core Algorithms

3.1 Step 1: Implication Elimination

3.1.1 Transformation Rule

The implication operator is eliminated using the logical equivalence:

$$A \rightarrow B \equiv \neg A \vee B$$

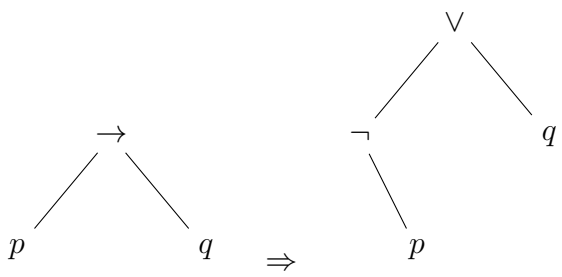
3.1.2 Algorithm

3.1.3 Example

Input: $(p \rightarrow q)$
Tree Transformation:

Algorithm 1 Implication Elimination

```
function EliminateImplication(node)
    if node = null then
        return null
    end if
    node.left ← EliminateImplication(node.left)
    node.right ← EliminateImplication(node.right)
    if node.data =→ then
        newNode ← new Node(∨)
        newNode.right ← node.right
        newNode.left ← new Node(¬)
        newNode.left.right ← node.left
        return newNode
    end if
    return node
end function
```



Output: $(\neg p \vee q)$

3.2 Step 2: Double Negation Removal

3.2.1 Transformation Rule

Double negations are eliminated using:

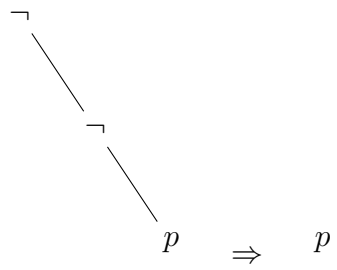
$$\neg\neg A \equiv A$$

3.2.2 Algorithm

3.2.3 Example

Input: $\neg\neg p$

Tree Transformation:



Output: p

Algorithm 2 Double Negation Removal

```
function RemoveDoubleNegation(node)
    if node = null then
        return null
    end if
    node.right ← RemoveDoubleNegation(node.right)
    node.left ← RemoveDoubleNegation(node.left)
    if node.data = ¬ and node.right ≠ null and node.right.data = ¬ then
        return node.right.right
    end if
    return node
end function
```

3.3 Step 3: Applying De Morgan's Laws

3.3.1 Transformation Rules

De Morgan's laws push negations through conjunctions and disjunctions:

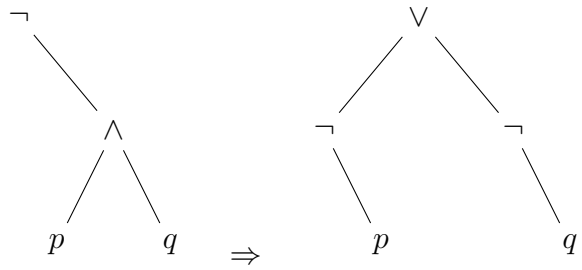
$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$
$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

3.3.2 Algorithm

3.3.3 Examples

Example 1: $\neg(p \wedge q)$

Tree Transformation:



Output: $(\neg p \vee \neg q)$

Example 2: $\neg(p \vee q)$

Output: $(\neg p \wedge \neg q)$

3.4 Step 4: Distribution of OR over AND

3.4.1 Transformation Rules

The distributive law is applied to move ORs inside ANDs:

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$
$$(A \wedge B) \vee C \equiv (A \vee C) \wedge (B \vee C)$$

Algorithm 3 Apply De Morgan's Laws

```
function ApplyDeMorgan(node)
    if node = null then
        return null
    end if
    node.right ← ApplyDeMorgan(node.right)
    node.left ← ApplyDeMorgan(node.left)
    if node.data = ¬ and node.right ≠ null and node.right.data ∈ {∧, ∨} then
        if node.right.data = ∧ then
            newNode ← new Node(∨)
            newNode.left ← new Node(¬)
            newNode.right ← new Node(¬)
            newNode.left.right ← node.right.left
            newNode.right.right ← node.right.right
            return newNode
        else if node.right.data = ∨ then
            newNode ← new Node(∧)
            newNode.left ← new Node(¬)
            newNode.right ← new Node(¬)
            newNode.left.right ← node.right.left
            newNode.right.right ← node.right.right
            return newNode
        end if
    end if
    return node
end function
```

Algorithm 4 Distribute OR over AND

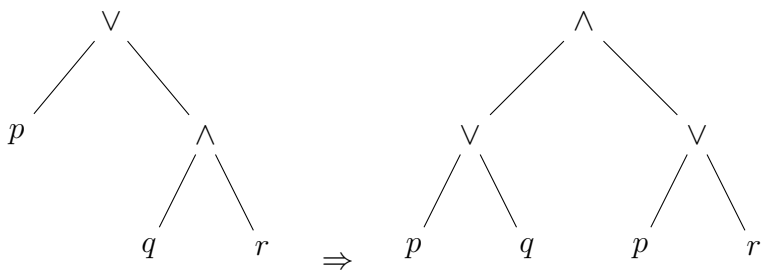
```
function DistributeOR(node)
    if node = null then
        return null
    end if
    node.left ← DistributeOR(node.left)
    node.right ← DistributeOR(node.right)
    if node.data = ∨ then
        if node.left ≠ null and node.left.data = ∧ then
            A ← node.left.left
            B ← node.left.right
            C ← node.right
            newNode ← new Node(∧)
            newNode.left ← new Node(∨)
            newNode.right ← new Node(∨)
            newNode.left.left ← A
            newNode.left.right ← C
            newNode.right.left ← B
            newNode.right.right ← C
            return DistributeOR(newNode)
        else if node.right ≠ null and node.right.data = ∧ then
            A ← node.left
            B ← node.right.left
            C ← node.right.right
            newNode ← new Node(∧)
            newNode.left ← new Node(∨)
            newNode.right ← new Node(∨)
            newNode.left.left ← A
            newNode.left.right ← B
            newNode.right.left ← A
            newNode.right.right ← C
            return DistributeOR(newNode)
        end if
    end if
    return node
end function
```

3.4.2 Algorithm

3.4.3 Examples

Example 1: $p \vee (q \wedge r)$

Tree Transformation:



Output: $(p \vee q) \wedge (p \vee r)$

Example 2: $(p \wedge q) \vee r$

Output: $(p \vee r) \wedge (q \vee r)$

4 Complete Conversion Process

4.1 Main Algorithm

The complete CNF conversion process applies transformations iteratively:

4.2 Iteration Strategy

- **Step 2-3**: Iterate up to 5 times or until no changes occur
  - Ensures all double negations are removed
  - Ensures all negations are pushed to literals
- **Step 4**: Iterate until fixed point
  - Distribution may create new opportunities for distribution
  - Continue until formula structure stabilizes

5 Formula Output

5.1 String Representation

The `CNF_formula()` function converts the tree back to string format with proper parenthesization:

```python
def CNF_formula(node: Optional[Node]) -> str:
    if node is None:
        return ""

    if node.left is None and node.right is None:
        return node.data
```

Algorithm 5 Convert to CNF

```
function ConvertToCNF(formula)
    tree ← ParseFormula(formula)
    Step 1: Eliminate implications
    tree ← EliminateImplication(tree)
    Step 2 & 3: Push negations inward (iterate 5 times)
    for i ← 1 to 5 do
        oldFormula ← TreeToString(tree)
        tree ← RemoveDoubleNegation(tree)
        tree ← ApplyDeMorgan(tree)
        tree ← RemoveDoubleNegation(tree)
        if oldFormula = TreeToString(tree) then
            break
        end if
    end for
    Step 4: Distribute OR over AND (until fixed point)
    while true do
        oldFormula ← TreeToString(tree)
        tree ← DistributeOR(tree)
        if oldFormula = TreeToString(tree) then
            break
        end if
    end while
    return TreeToString(tree)
end function
```

```python

    if node.data == NOT:
        if node.right and node.right.data in {OR, AND, IMP}:
            return f"¬({CNF_formula(node.right)})"
        return NOT + CNF_formula(node.right)

    left_str = CNF_formula(node.left)
    right_str = CNF_formula(node.right)

    if node.data == OR:
        if node.left and node.left.data == AND:
            left_str = f"({left_str})"
        if node.right and node.right.data == AND:
            right_str = f"({right_str})"
        return f"{left_str}∨{right_str}"

    if node.data == AND:
        if node.left and node.left.data == OR:
            left_str = f"({left_str})"
        if node.right and node.right.data == OR:
            right_str = f"({right_str})"
        return f"{left_str}∧{right_str}"

    return f"({left_str}{node.data}{right_str})"
```

Listing 2: Formula string generation

5.2 Parenthesization Rules

- Parentheses are added when a lower-precedence operator is child of higher-precedence operator
- For OR nodes: Parenthesize AND children
- For AND nodes: Parenthesize OR children
- For NOT nodes: Parenthesize complex expressions

6 Complete Examples

6.1 Example 1: Simple Implication

Input: $(p \to q)$

Step-by-step transformation:

1. Eliminate implication: $(\neg p \vee q)$
2. No double negations
3. No De Morgan's needed
4. No distribution needed

Final CNF: $(\neg p \vee q)$

6.2 Example 2: Complex Formula

Input: $(p \to (q \land r))$

Step-by-step transformation:

1. Eliminate implication: $(\neg p \lor (q \land r))$
2. Distribute OR over AND: $(\neg p \lor q) \land (\neg p \lor r)$

Final CNF: $(\neg p \lor q) \land (\neg p \lor r)$

6.3 Example 3: De Morgan's Application

Input: $\neg(p \land q) \lor r$

Step-by-step transformation:

1. No implication
2. Apply De Morgan's: $(\neg p \lor \neg q) \lor r$
3. Simplification (associativity): Already in CNF form

Final CNF: $(\neg p \lor \neg q \lor r)$

6.4 Example 4: Nested Formula

Input: $((p \to q) \to r)$

Step-by-step transformation:

1. Eliminate outer implication: $(\neg(p \to q) \lor r)$
2. Eliminate inner implication: $(\neg(\neg p \lor q) \lor r)$
3. Apply De Morgan's: $((\neg\neg p \land \neg q) \lor r)$
4. Remove double negation: $((p \land \neg q) \lor r)$
5. Distribute: $(p \lor r) \land (\neg q \lor r)$

Final CNF: $(p \lor r) \land (\neg q \lor r)$

7 Complexity Analysis

7.1 Time Complexity

- **Implication Elimination**: $O(n)$ - single tree traversal
- **Double Negation Removal**: $O(n)$ per iteration, up to 5 iterations: $O(n)$
- **De Morgan's Application**: $O(n)$ per iteration, up to 5 iterations: $O(n)$
- **Distribution**: $O(n)$ per iteration, up to $O(n)$ iterations in worst case: $O(n^2)$
  - Each distribution can double tree size in worst case
  - Maximum $\log n$ doublings before stabilization
  - Actual worst case: $O(2^n)$ for deeply nested formulas

Overall worst case: $O(2^n)$ where n is the formula size

7.2 Space Complexity

- **Tree Storage**: $O(n)$ initially, can grow to $O(2^n)$ after distribution
- **Recursion Stack**: $O(h)$ where h is tree height
- **Total**: $O(2^n)$ in worst case

7.3 Optimization Notes

The exponential worst case occurs with formulas like:

$$(a_1 \vee b_1) \wedge (a_2 \vee b_2) \wedge \ldots \wedge (a_n \vee b_n)$$

Which is already in CNF. The algorithm handles this efficiently by detecting no changes in the distribution step.

8 Usage Instructions

8.1 Input Format

Same as Phase 1:

- Lowercase letters for variables: `p, q, r, s, ...`
- Unicode symbols for operators: ¬, ∧, ∨, →
- Parentheses for grouping
- Spaces are ignored

8.2 Running the Program

```
# Run the program
python cnf_converter.py

# Enter filename when prompted
Please enter a file name:
formula.txt

# Output shows the CNF result
====================
(¬p∨q)∧(¬p∨r)
====================
```

Listing 3: Execution example

8.3 Example Input Files

File 1: simple.txt

```
(p -> q)
```

File 2: complex.txt

```
((p ^ q) -> (r v s))
```

File 3: nested.txt

```
~((p -> q) ^ (q -> r))
```

9 Correctness and Soundness

9.1 Logical Equivalence

Each transformation preserves logical equivalence:

[Transformation Correctness] For any formula ϕ, if ψ is obtained from ϕ by applying the CNF conversion algorithm, then $\phi \equiv \psi$ (logically equivalent).

[Proof Sketch] Each step applies a logical equivalence:

1. Implication elimination: $A \to B \equiv \neg A \vee B$ (by definition)
2. Double negation: $\neg\neg A \equiv A$ (law of double negation)
3. De Morgan's laws: Proven logical equivalences
4. Distribution: $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$ (distributive law)

Therefore, the composition of these transformations preserves equivalence.

9.2 Termination

[Algorithm Termination] The CNF conversion algorithm always terminates.

[Proof Sketch]

1. Steps 1-3 perform a bounded number of iterations (5 maximum)
2. Step 4 terminates because:
   - Each distribution strictly increases the number of AND nodes at the top level
   - There is a maximum depth of nesting
   - The algorithm detects when no changes occur and exits

10 Limitations and Extensions

10.1 Current Limitations

1. Formula size can grow exponentially during distribution
2. No optimization for already-CNF formulas
3. No clause simplification (e.g., $(p \vee p) \equiv p$)
4. No tautology elimination (e.g., $(p \vee \neg p)$ always true)

10.2 Potential Enhancements

1. **Early CNF detection**: Check if formula is already in CNF
2. **Clause simplification**: Remove duplicate literals, tautologies
3. **Formula minimization**: Use Quine-McCluskey or similar algorithms
4. **DIMACS format output**: For SAT solver integration
5. **Tseitin transformation**: Avoid exponential blowup using auxiliary variables
6. **DNF support**: Implement Disjunctive Normal Form conversion
7. **Equivalence support**: Handle biconditional operator ($\leftrightarrow$)

11 Applications

CNF conversion is fundamental in:

- **Automated Theorem Proving**: Resolution requires CNF formulas
- **SAT Solvers**: Most SAT solvers expect CNF input
- **Model Checking**: Temporal logic model checkers use CNF
- **Circuit Design**: Logic synthesis and verification
- **AI Planning**: STRIPS and other planning systems

12 Conclusion

This CNF converter implements a systematic approach to transforming propositional logic formulas into Conjunctive Normal Form. The implementation correctly applies standard logical transformations while maintaining logical equivalence.

The modular design separates concerns: parsing builds the initial tree, individual transformation functions handle specific logical rules, and the main algorithm coordinates the overall conversion process. This separation enables easy debugging, testing, and extension.

While the worst-case complexity is exponential, the algorithm performs efficiently on typical formulas and serves as a solid foundation for integration with automated reasoning systems.

A Complete Source Code

```python
import sys
from typing import Optional

# Define logical symbols used in formula
NOT = '¬'
```

```python
AND = '∧'
OR = '∨'
IMP = '→'

# operator precedence
OPERATOR_PRECEDENCE = {
    IMP: 1, OR: 2, AND: 3, NOT: 4
}

# Node class definition
class Node:
    def __init__(self, key):
        self.data = key
        self.left: Node | None = None
        self.right: Node | None = None

# Step 1: Elimination of implication
def elimination_IMP(node: Optional[Node]) -> Optional[Node]:
    if node is None:
        return None

    node.left = elimination_IMP(node.left)
    node.right = elimination_IMP(node.right)

    if node.data == IMP:
        new_node = Node(OR)
        new_node.right = node.right
        new_node.left = Node(NOT)
        new_node.left.right = node.left
        return new_node
    return node

# Step 2: Remove double negation
def remove_double_negation(node: Optional[Node]) -> Optional[Node
    ]:
    if node is None:
        return None

    node.right = remove_double_negation(node.right)
    node.left = remove_double_negation(node.left)

    if (node.data == NOT and node.right is not None
        and node.right.data == NOT):
        return node.right.right

    return node

# Step 3: Applying De Morgan's law
def apply_demorgan(node: Optional[Node]) -> Optional[Node]:
    if node is None:
        return None
```

```python

    node.right = apply_demorgan(node.right)
    node.left = apply_demorgan(node.left)

    if (node.data == NOT and node.right is not None
        and node.right.data in {OR, AND}):

        if node.right.data == AND:
            new_node = Node(OR)
            new_node.left = Node(NOT)
            new_node.right = Node(NOT)
            new_node.left.right = node.right.left
            new_node.right.right = node.right.right
            return new_node

        elif node.right.data == OR:
            new_node = Node(AND)
            new_node.left = Node(NOT)
            new_node.right = Node(NOT)
            new_node.left.right = node.right.left
            new_node.right.right = node.right.right
            return new_node

    return node

# Step 4: Distribute OR over AND
def OR_over_AND(node: Optional[Node]) -> Optional[Node]:
    if node is None:
        return None

    node.left = OR_over_AND(node.left)
    node.right = OR_over_AND(node.right)

    if node.data == OR:
        if node.left and node.left.data == AND:
            temp1 = node.left.left
            temp2 = node.left.right
            temp3 = node.right

            new_node = Node(AND)
            new_node.right = Node(OR)
            new_node.left = Node(OR)

            new_node.left.left = temp1
            new_node.left.right = temp3

            new_node.right.left = temp2
            new_node.right.right = temp3

            return OR_over_AND(new_node)

```

```python
        elif node.right is not None and node.right.data == AND:
            temp1 = node.left
            temp2 = node.right.left
            temp3 = node.right.right

            new_node = Node(AND)
            new_node.right = Node(OR)
            new_node.left = Node(OR)

            new_node.left.right = temp2
            new_node.left.left = temp1

            new_node.right.right = temp3
            new_node.right.left = temp1

            return OR_over_AND(new_node)

    return node

# Formula string generation
def CNF_formula(node: Optional[Node]) -> str:
    if node is None:
        return ""

    if node.left is None and node.right is None:
        return node.data

    if node.data == NOT:
        if node.right and node.right.data in {OR, AND, IMP}:
            return f"¬({CNF_formula(node.right)})"
        return NOT + CNF_formula(node.right)

    left_str = CNF_formula(node.left)
    right_str = CNF_formula(node.right)

    if node.data == OR:
        if node.left and node.left.data == AND:
            left_str = f"({left_str})"
        if node.right and node.right.data == AND:
            right_str = f"({right_str})"
        return f"{left_str}∨{right_str}"

    if node.data == AND:
        if node.left and node.left.data == OR:
            left_str = f"({left_str})"
        if node.right and node.right.data == OR:
            right_str = f"({right_str})"
        return f"{left_str}∧{right_str}"

    return f"({left_str}{node.data}{right_str})"

```

```python
# Main conversion function
def main():
    formula = get_input()

    if not check_parentheses(formula):
        print("Invalid input!")
        sys.exit(1)

    tree = binary_tree(formula, 0, len(formula)-1)
    tree = elimination_IMP(tree)

    # Apply transformations iteratively
    for _ in range(5):
        old_tree_str = CNF_formula(tree)
        tree = remove_double_negation(tree)
        tree = apply_demorgan(tree)
        tree = remove_double_negation(tree)
        if old_tree_str == CNF_formula(tree):
            break

    while True:
        old_tree_str = CNF_formula(tree)
        tree = OR_over_AND(tree)
        if old_tree_str == CNF_formula(tree):
            break

    print("====================")
    print(CNF_formula(tree))
    print("====================")

if __name__ == "__main__":
    main()
```

Listing 4: Complete CNF Converter Implementation